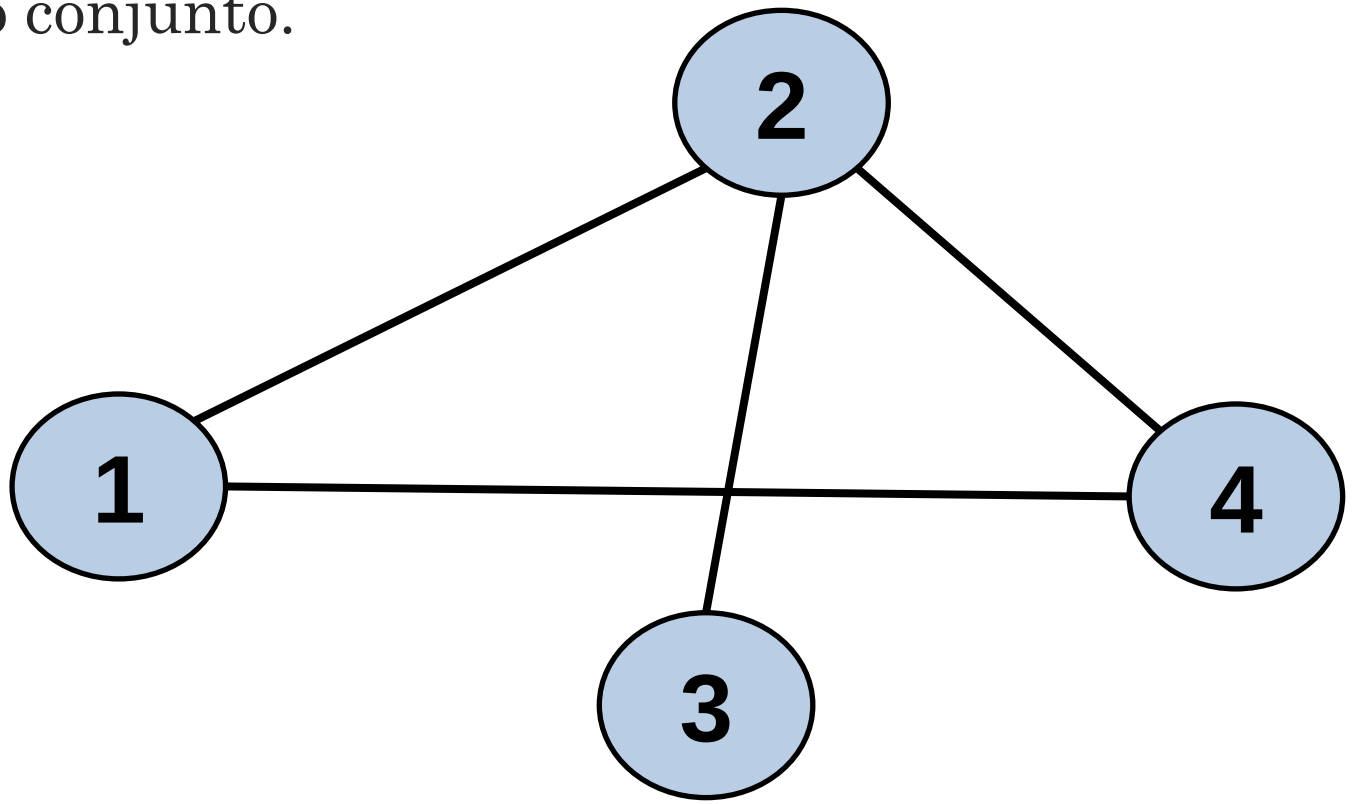
ESTRUTURA DE DADOS: GRAFOS

Definições e conceitos

Prof. Jean Nunes

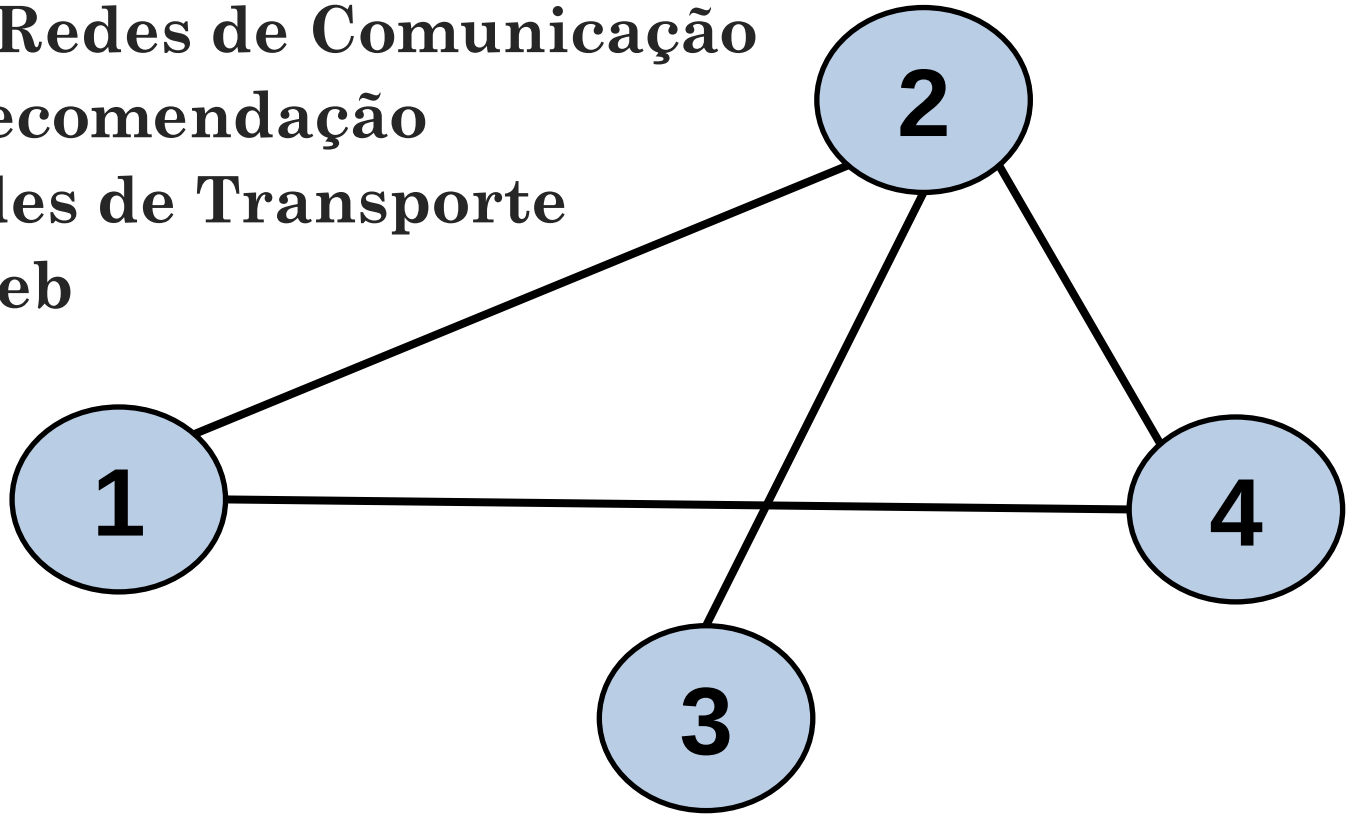
Definição

- Como representar um conjunto de objetos e as suas relações?
  - Diversos tipos de aplicações necessitam disso
  - É um modelo matemático que representa as relações entre objetos de um determinado conjunto.



Definição

- Grafos em computação
  - Forma de solucionar problemas computáveis
  - Buscam o desenvolvimento de algoritmos mais eficientes
    - **Redes Sociais**
    - **Roteamento e Redes de Comunicação**
    - **Sistemas de Recomendação**
    - **Análise de Redes de Transporte**
    - **Pesquisa na Web**



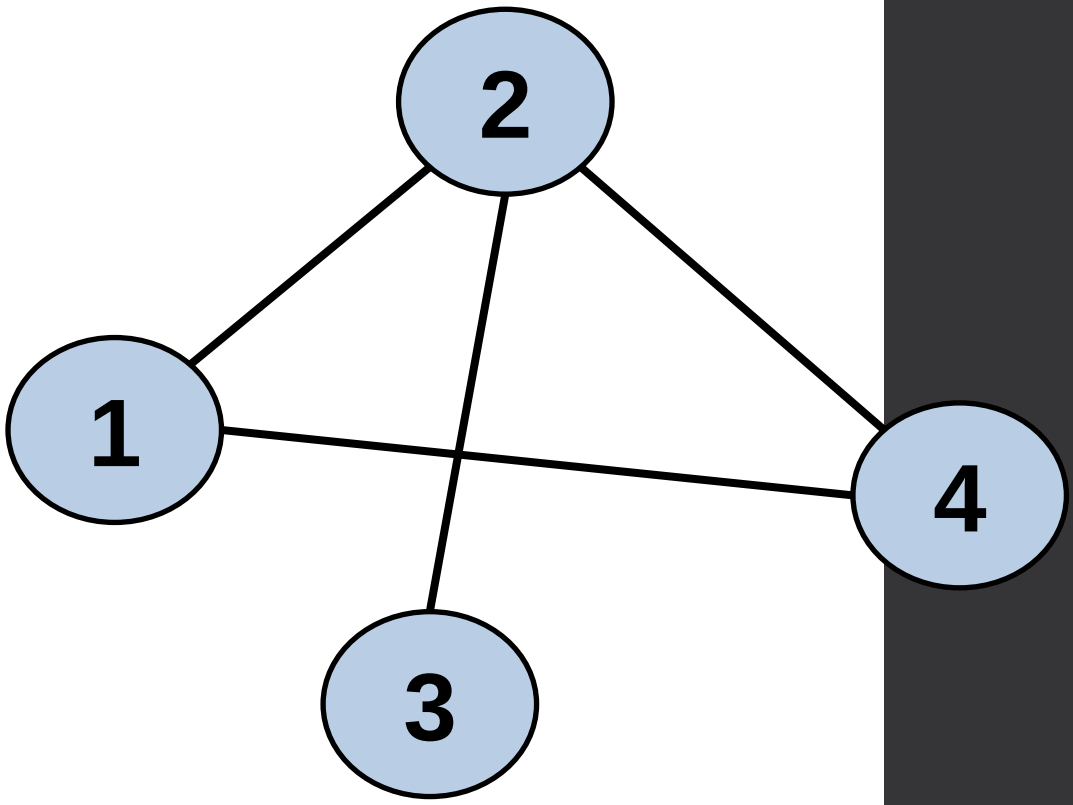
Definição

- Um grafo G(V,A) é definido por dois conjuntos
  - Conjunto V de vértices (não vazio)
    - Itens representados em um grafo;
  - Conjunto A de arestas
    - Utilizadas para conectar pares de vértices, usando um critério previamente estabelecido.

G(V,A)

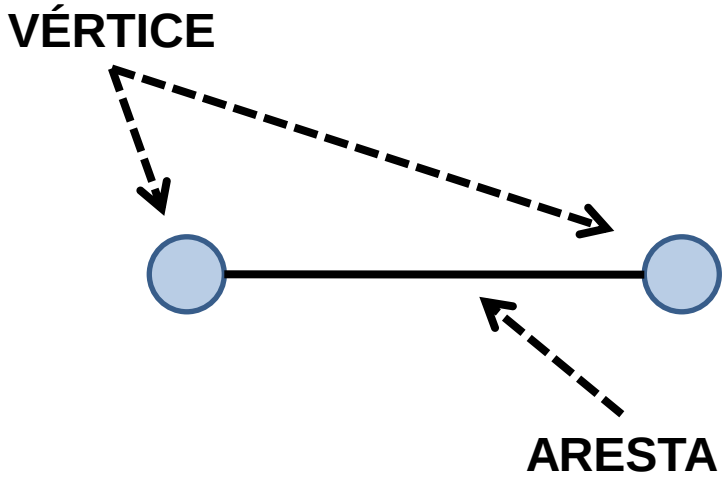
V = {1,2,3,4}

A = {{1,2},{1,4},{2,3},{2,4}}



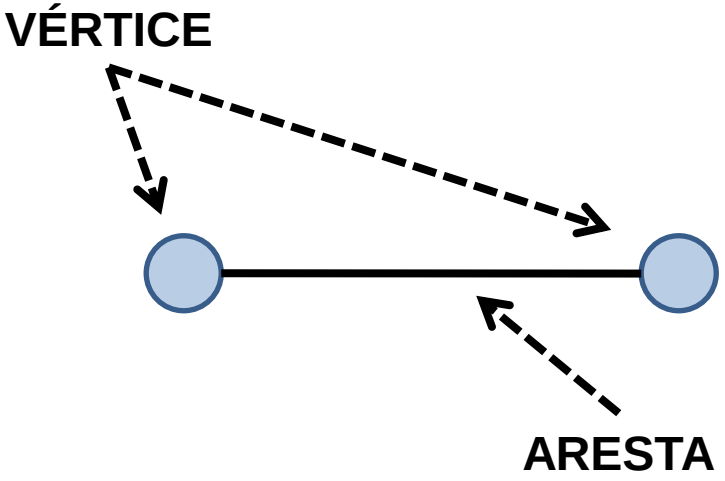
Conceitos básicos

- Vértice é cada um dos itens representados no grafo.
  - O seu significado depende da natureza do problema modelado
    - Pessoas, uma tarefa em um projeto, lugares em um mapa, etc.



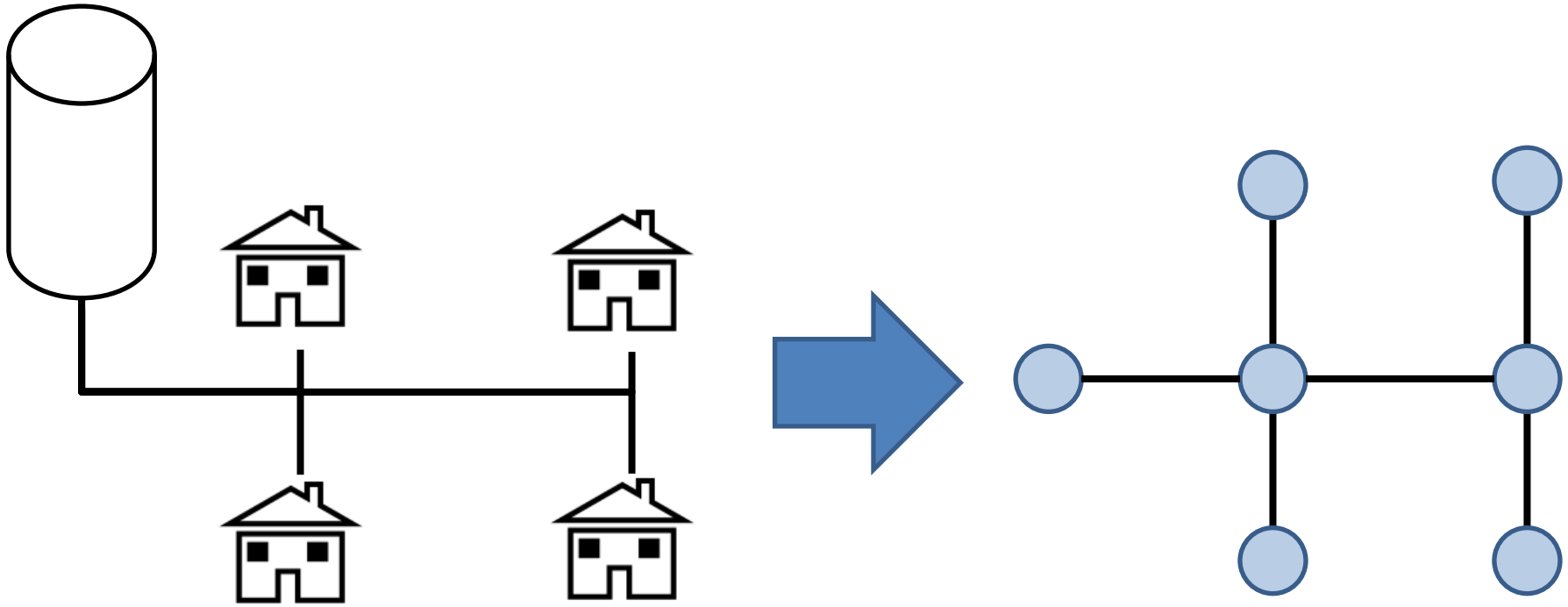
Conceitos básicos

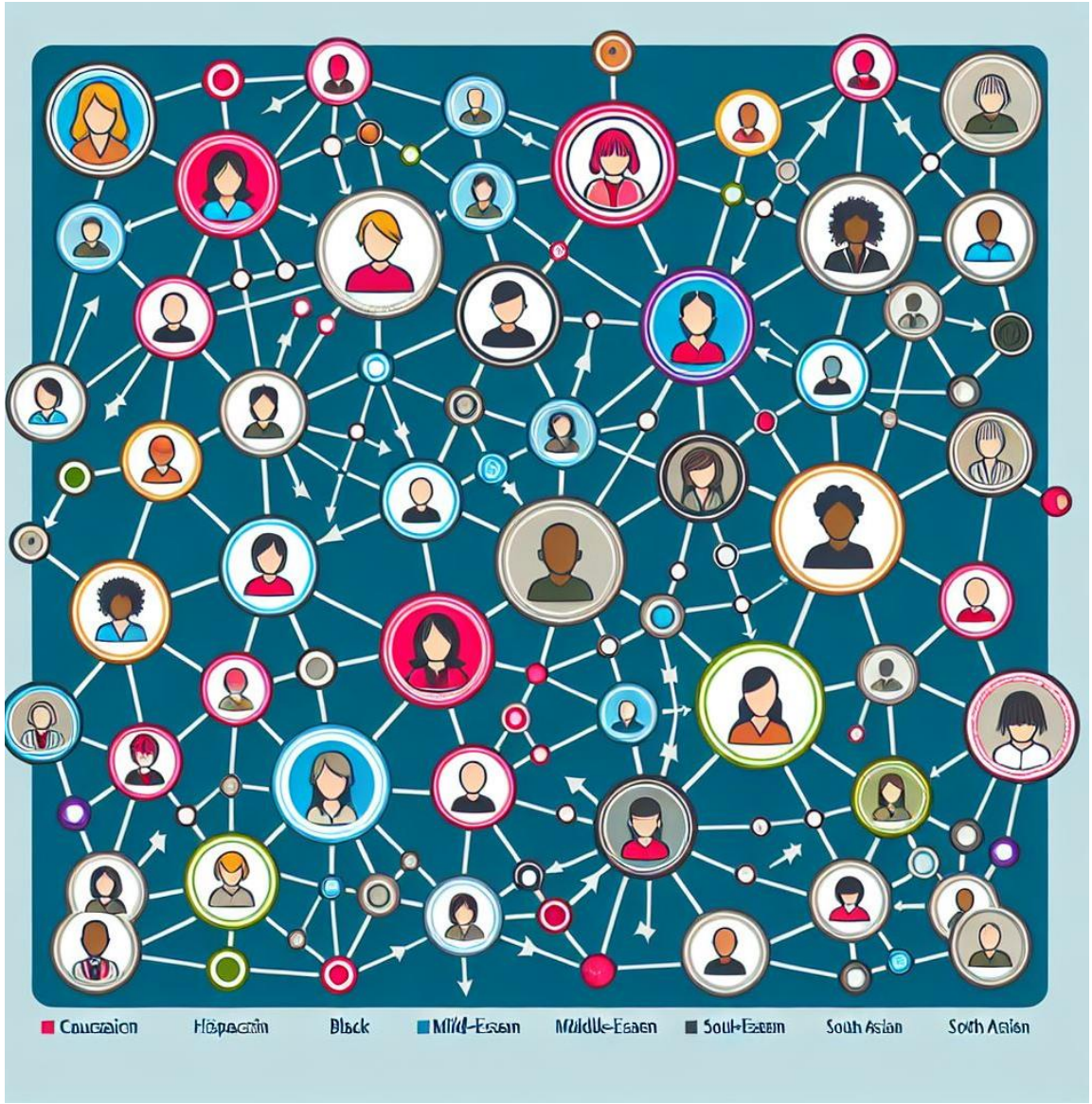
- Aresta (ou arco) liga dois vértices
  - Diz qual a relação entre eles
  - Dois vértices são **adjacentes** se existir uma aresta ligando eles.
    - Pessoas (parentesco entre elas ou amizade), tarefas de um projeto (pré-requisito entre as tarefas), lugares de um mapa (estradas que existem ligando os lugares), etc.



Conceitos básicos

- Praticamente qualquer objeto pode ser representado como um grafo.
  - Exemplo: sistema de distribuição de água



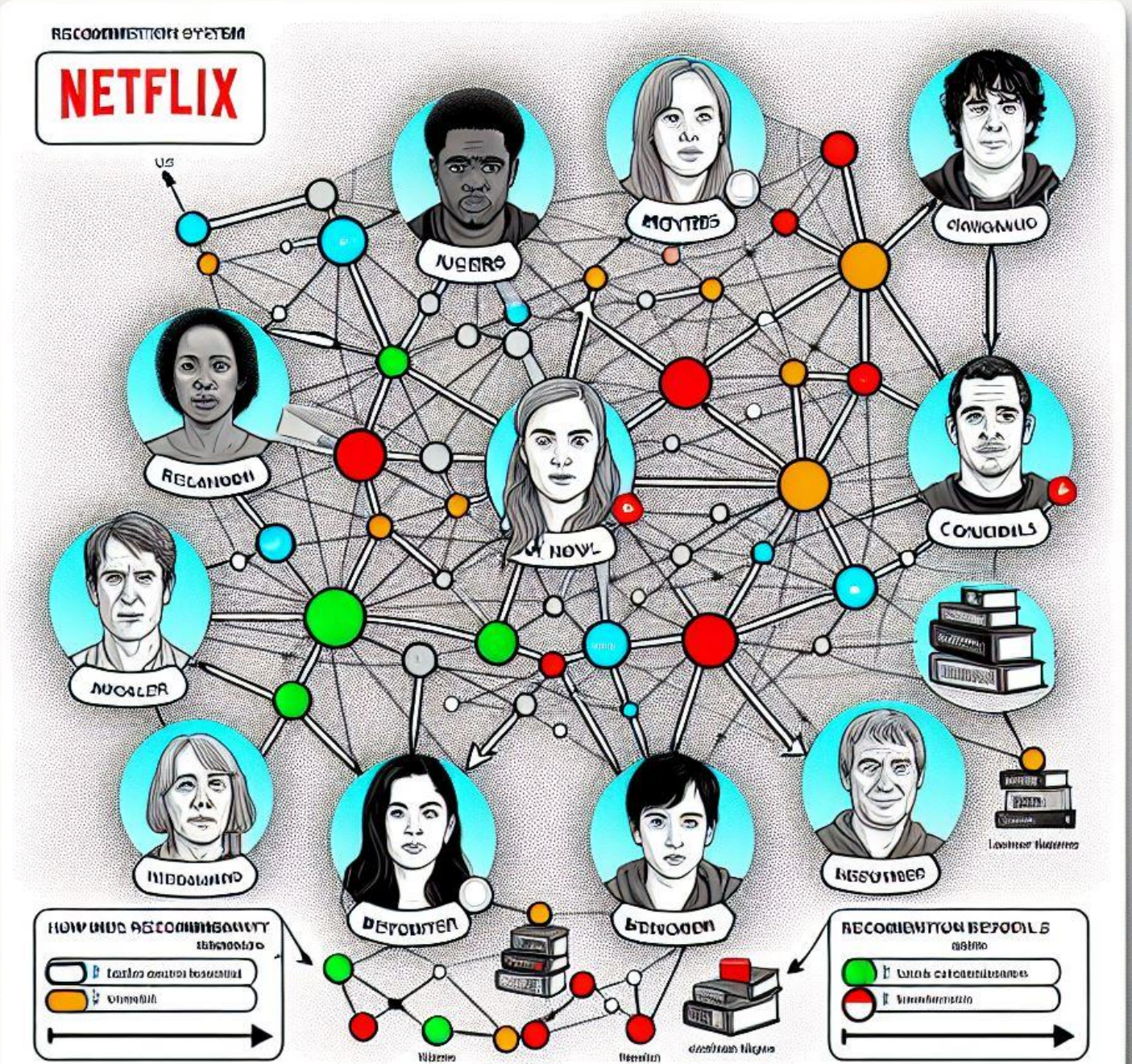


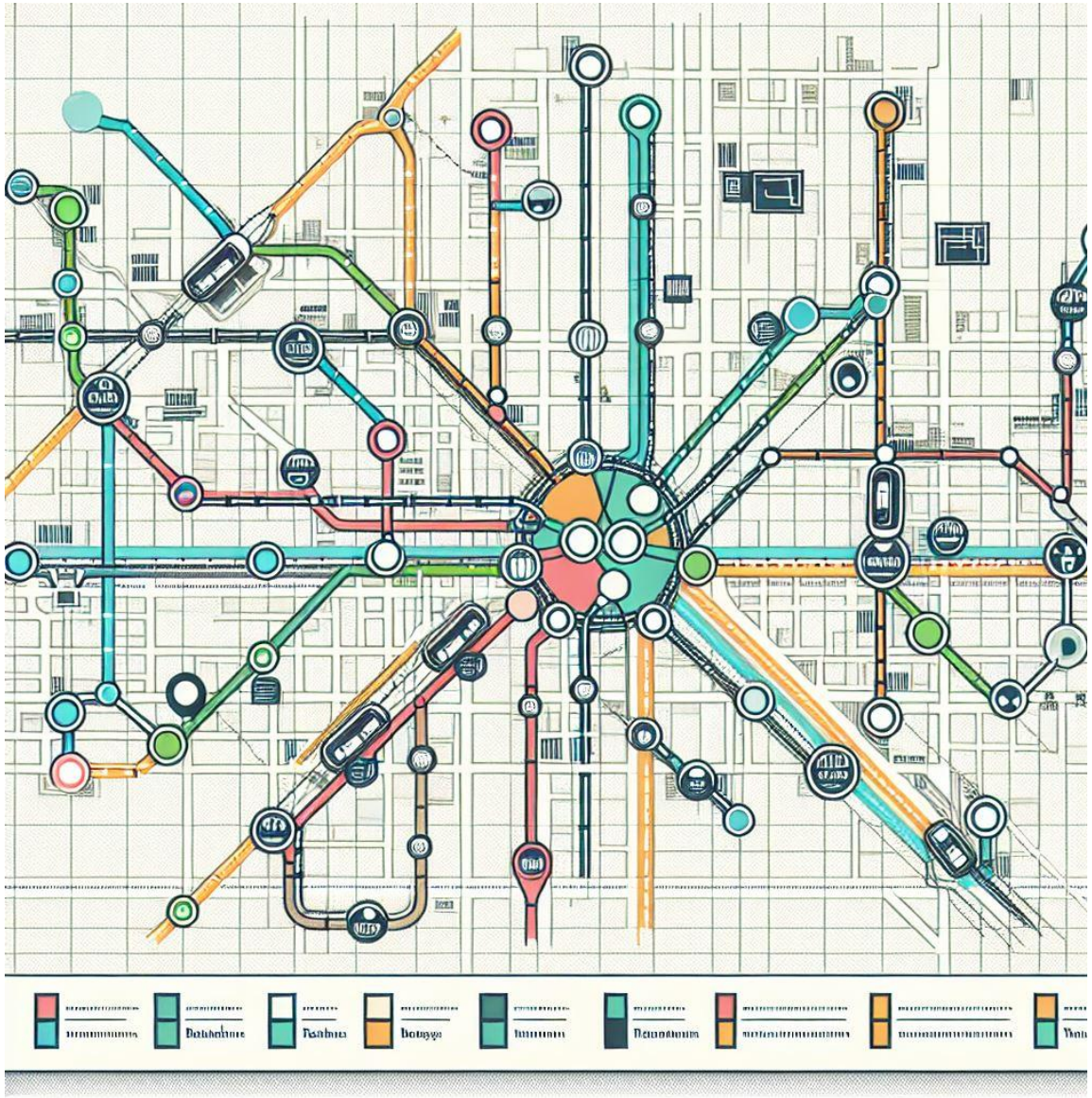
Conceitos básicos

- Praticamente qualquer objeto pode ser representado como um grafo
  - Exemplo: rede social

Conceitos básicos

- Praticamente qualquer objeto pode ser representado como um grafo
  - Exemplo: sistema de recomendação



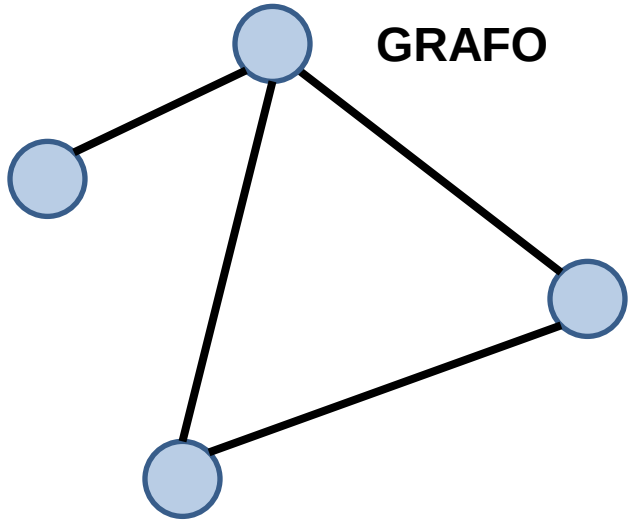


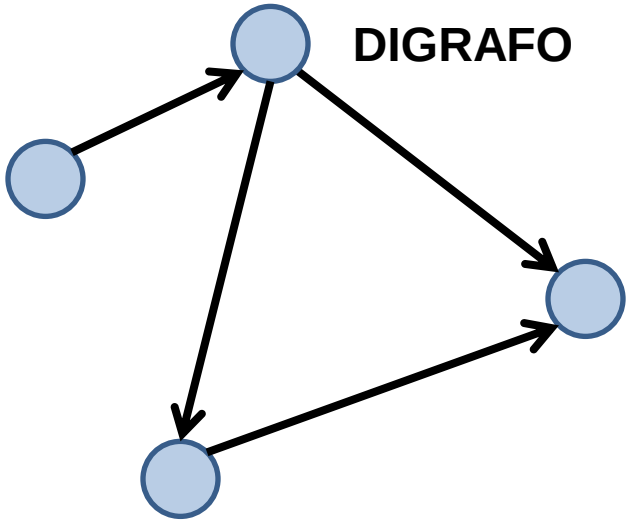
Conceitos básicos

- Praticamente qualquer objeto pode ser representado como um grafo
  - Exemplo: análise de sistema de transporte

Conceitos básicos

- As arestas podem ou não ter direção
  - Existe um orientação quanto ao sentido da aresta
    - Em um grafo direcionado ou **digrafo**, se uma aresta liga os vértices **A** a **B**, isso significa que podemos ir de **A** para **B**, mas não o contrário



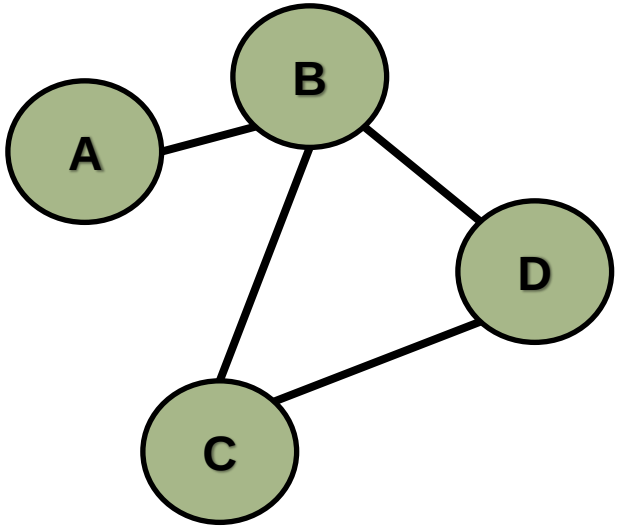


Conceitos básicos

- Grau
  - Indica o número de arestas que conectam um vértice do grafo a outros vértices
    - número de vizinhos que aquele vértice possui no grafo (que chegam ou partem dele)
  - No caso dos dígrafos, temos dois tipos de grau:
    - **grau de entrada**: número de arestas que chegam ao vértice;
    - **grau de saída**: número de arestas que partem do vértice.

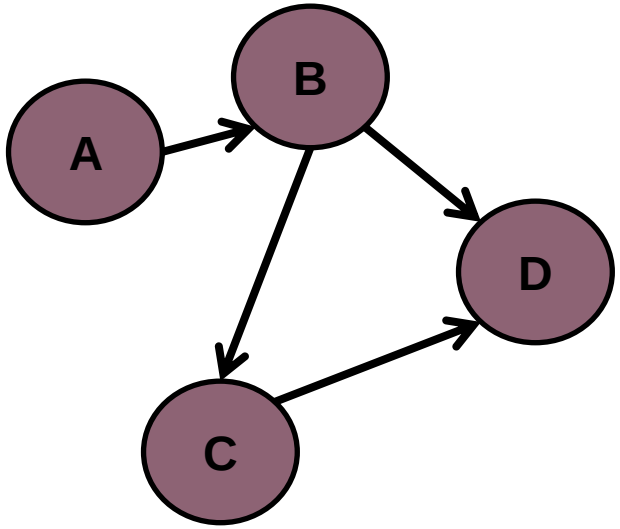
Conceitos básicos

GRAFO



Grau
G(A) = 1
G(B) = 3
G(C) = 2
G(D) = 2

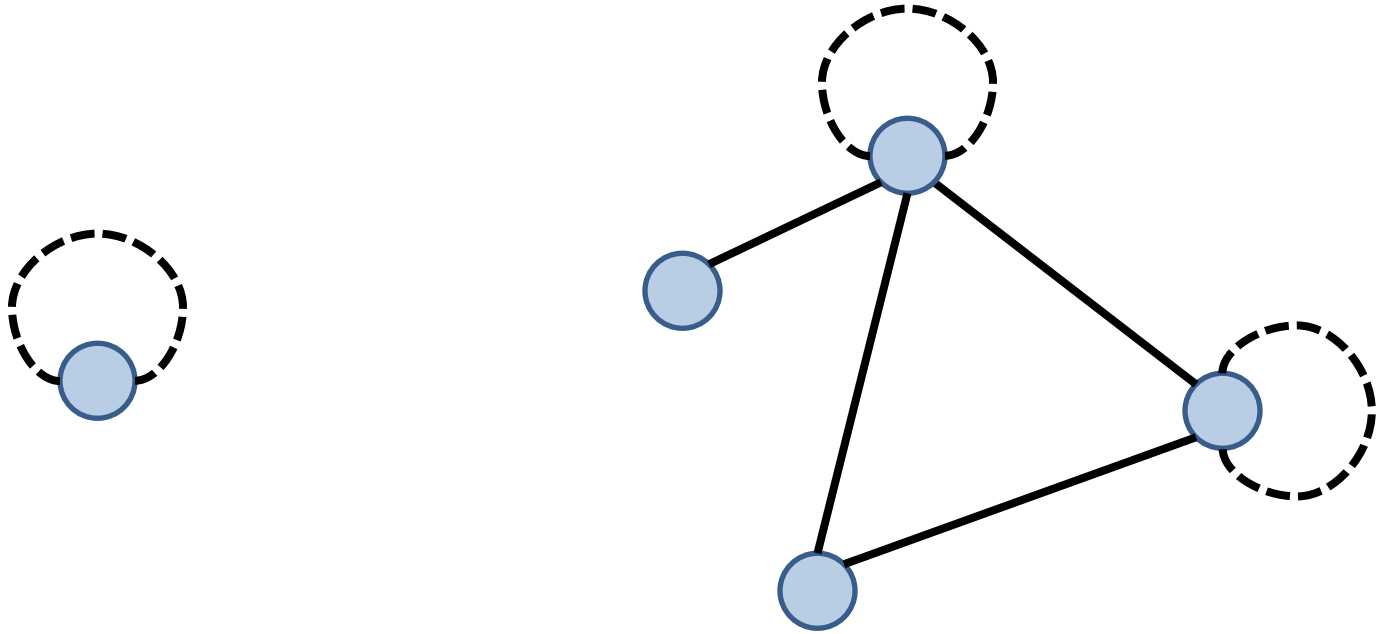
DIGRAFO



Grau Entrada	Grau Saída
G(A) = 0	G(A) = 1
G(B) = 1	G(B) = 2
G(C) = 1	G(C) = 1
G(D) = 2	G(D) = 0

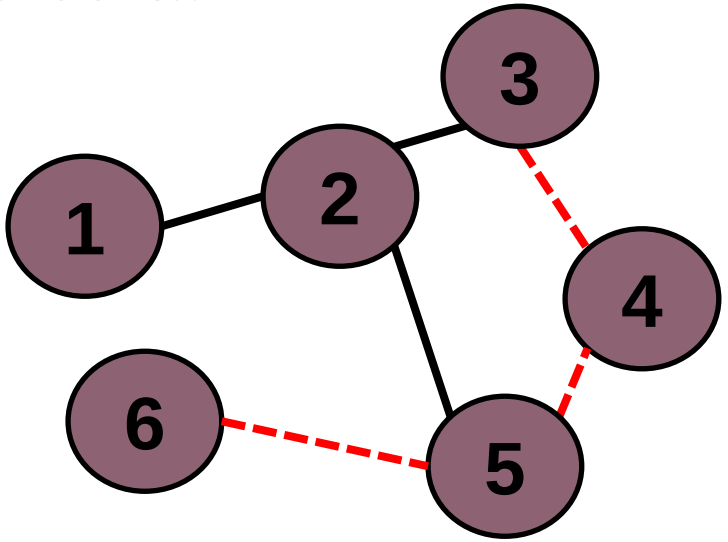
Conceitos básicos

- Laço
  - Uma aresta é chamada de laço se seu vértice de partida é o mesmo que o de chagada
    - A aresta conecta o vértice a ele mesmo

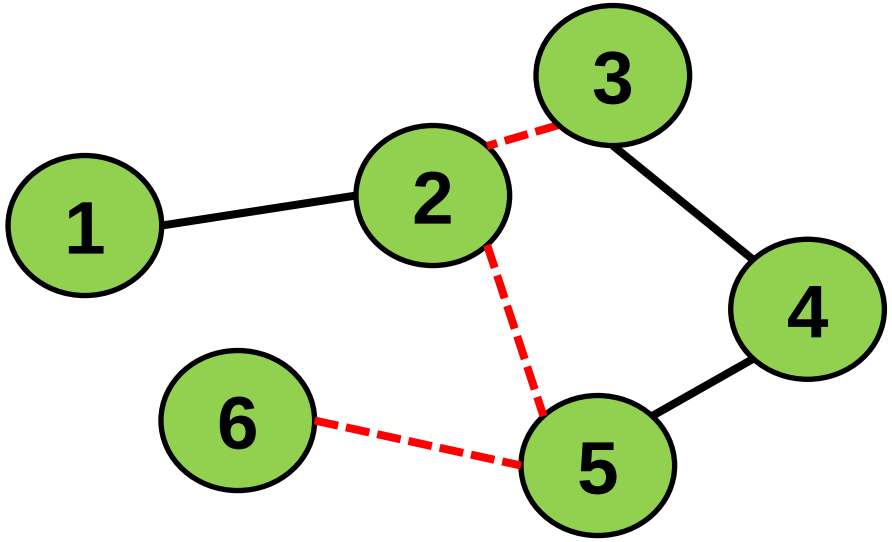


Conceitos básicos

- Caminho
  - Um caminho entre dois vértices é uma sequência de vértices onde cada vértice está conectado ao vértice seguinte por meio de uma aresta.



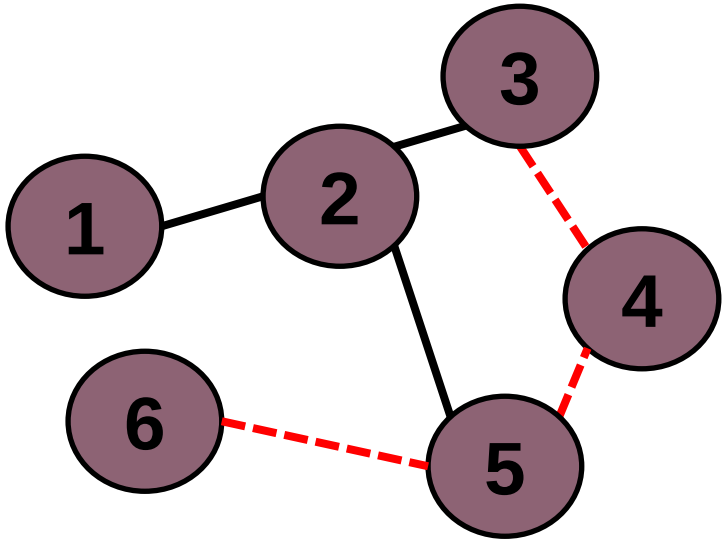
CAMINHO: 3-4-5-6



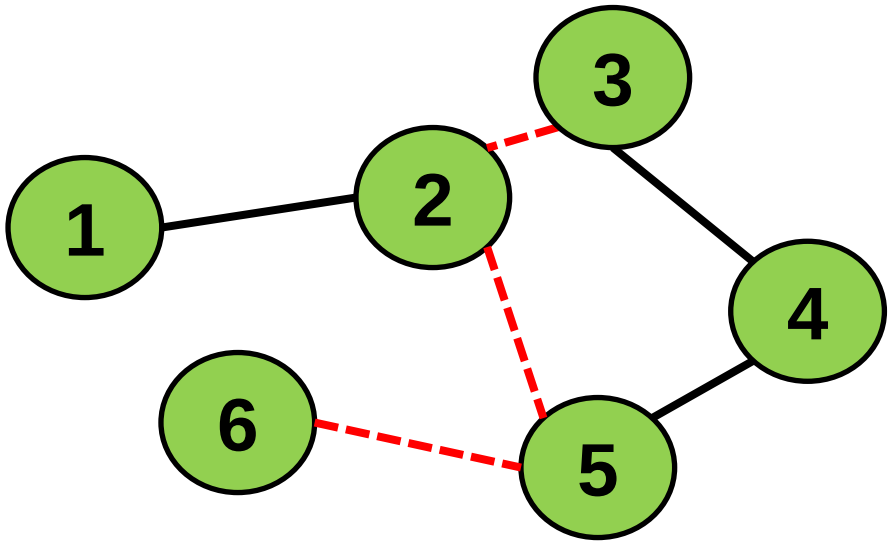
CAMINHO: 3-2-5-6

Conceitos básicos

- Caminho
  - **Comprimento do caminho:** número de vértices que precisamos percorrer de um vértice até o outro



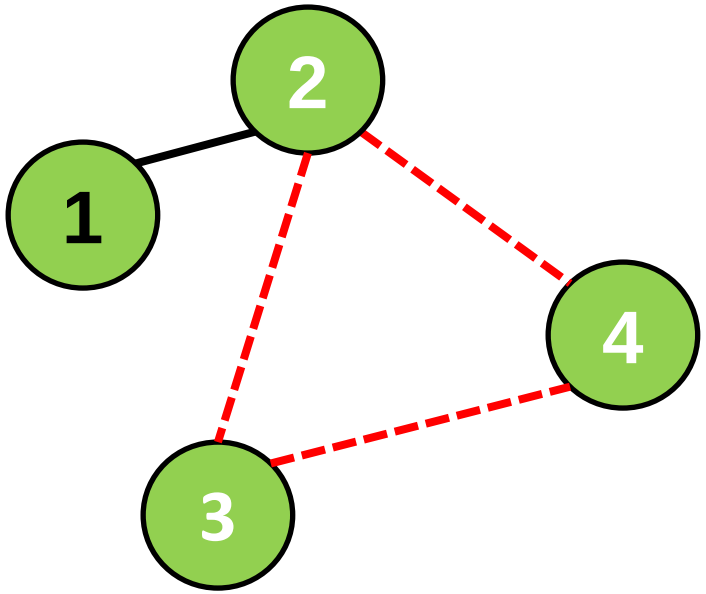
CAMINHO: 3-4-5-6



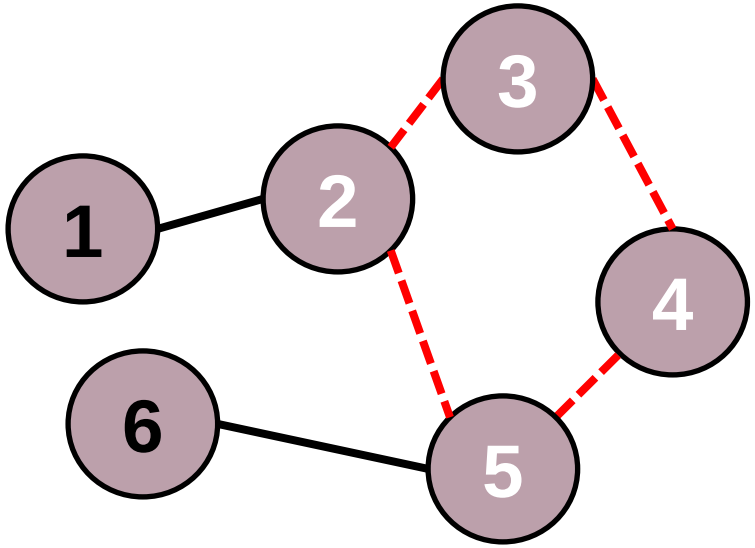
CAMINHO: 3-2-5-6

Conceitos básicos

- **Ciclo**
  - Caminho onde o vértice inicial e o final são o mesmo vértice.
    - Note que um ciclo é um caminho fechado sem vértices repetidos



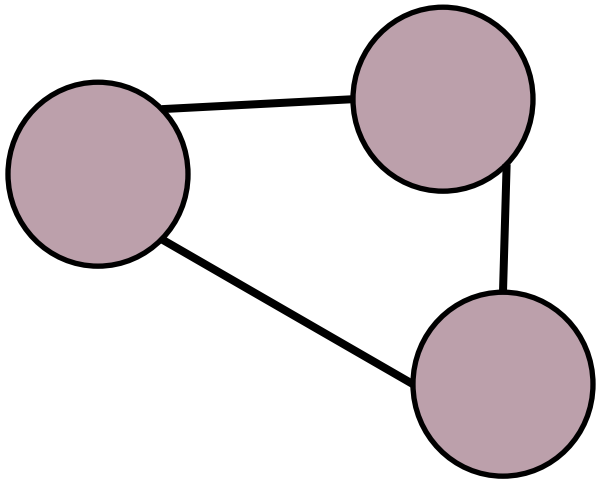
CICLO: 2-3-4

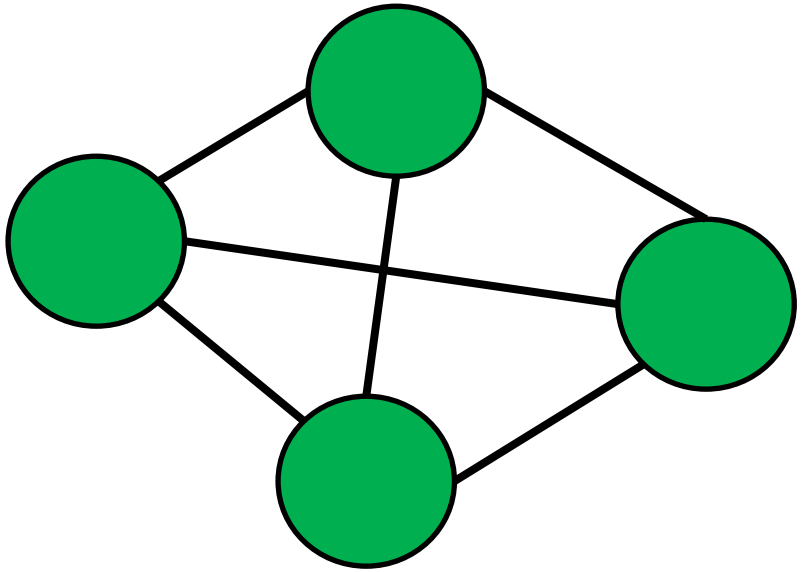


CICLO: 2-3-4-5

Tipos de Grafos

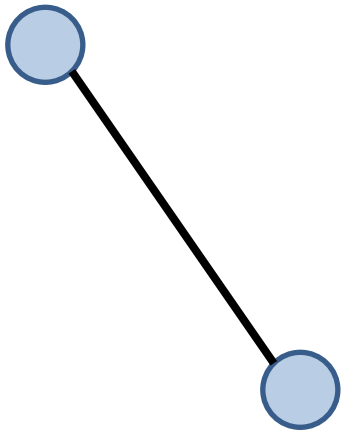
- Grafo completo
  - Grafo simples onde cada vértice se conecta a todos os outros vértices do grafo.



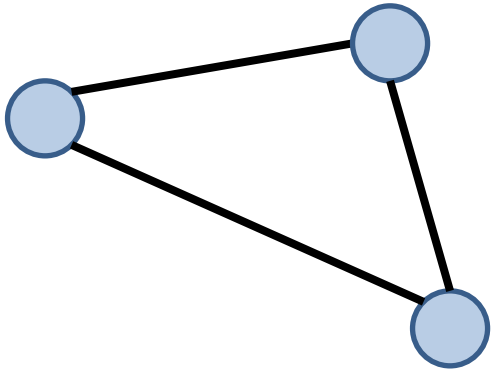


Tipos de Grafos

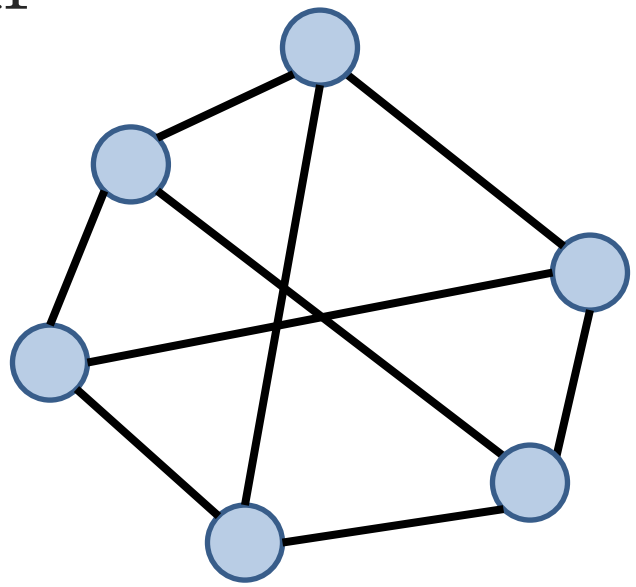
- Grafo regular
  - Grafo onde todos os seus vértices possuem o mesmo grau (número de arestas ligadas a ele)
  - Todo grafo completo é também regular



Grau = 1



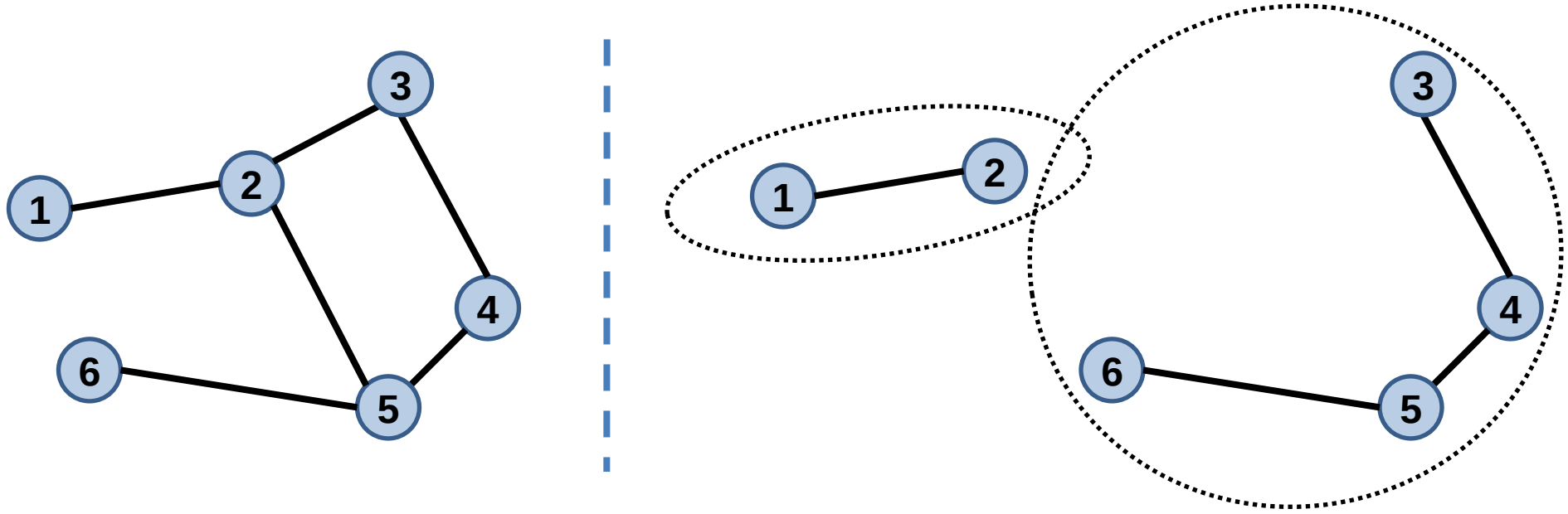
Grau = 2



Grau = 3

Tipos de Grafos

- Subgrafo
  - Gs(Vs,As) é um subgrafo de G(V,A) se o conjunto de vértices Vs for um subconjunto de V, Vs ⊆ V, e se o conjunto de arestas As for um subconjunto de A, As ⊆ A.

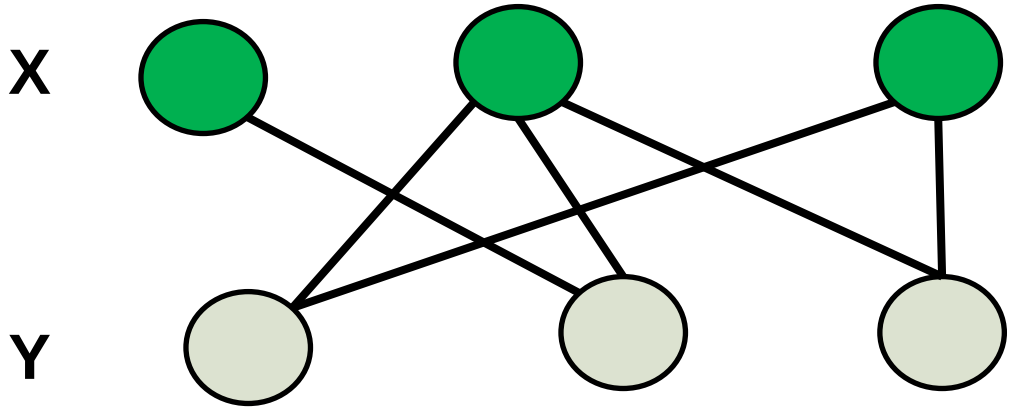


GRAFO

SUBGRAFOS

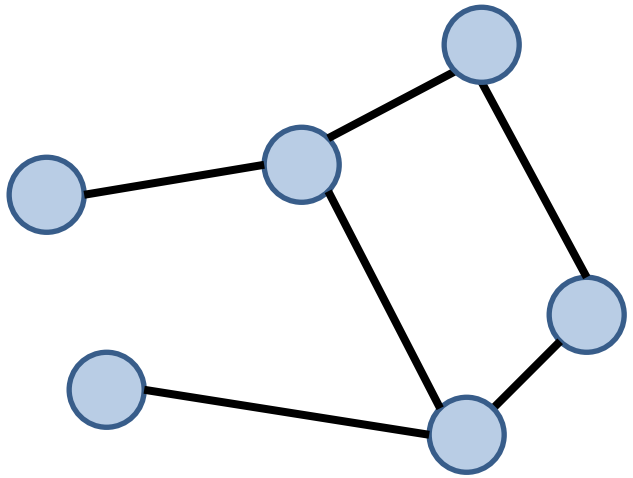
Tipos de Grafos

- Grafo bipartido
  - Um grafo G(V,A) onde o seu conjunto de vértices pode ser divididos em dois subconjuntos X e Y sem intersecção.
    - As arestas conectam apenas os vértices que estão em subconjuntos diferentes

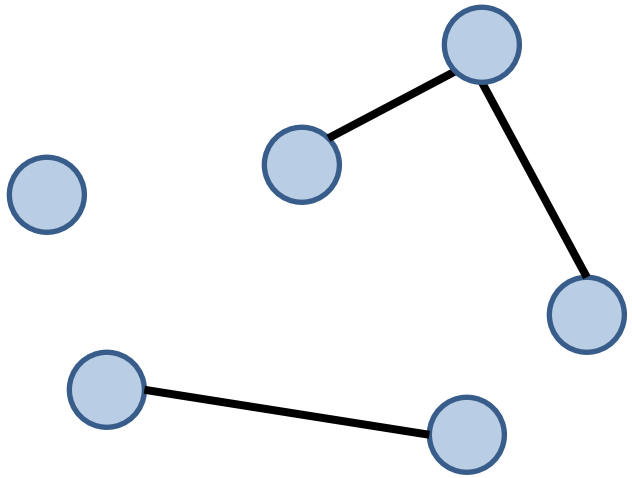


Tipos de Grafos

- Grafo conexo e desconexo
  - **Grafo conexo**: existe um caminho ligando quaisquer dois vértices.
  - Quando isso não acontece, temos um **grafo desconexo**



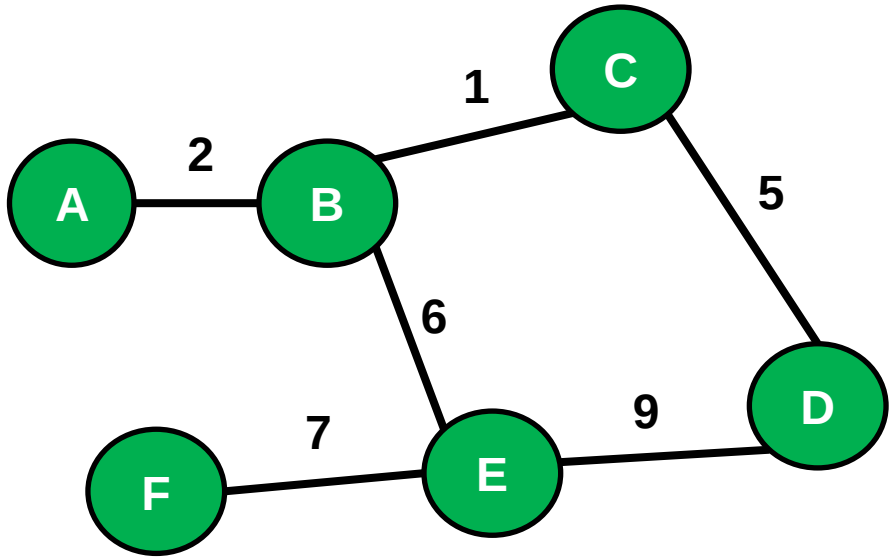
GRAFO CONEXO



GRAFO DESCONEXO

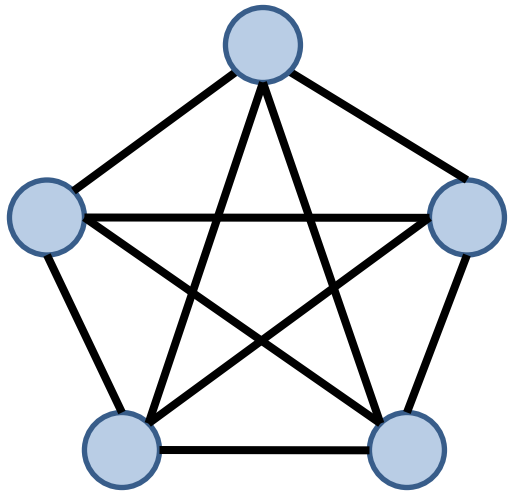
Tipos de Grafos

- Grafo ponderado
  - É um grafo que possui **pesos** (valor numérico) associados a cada uma de suas arestas.

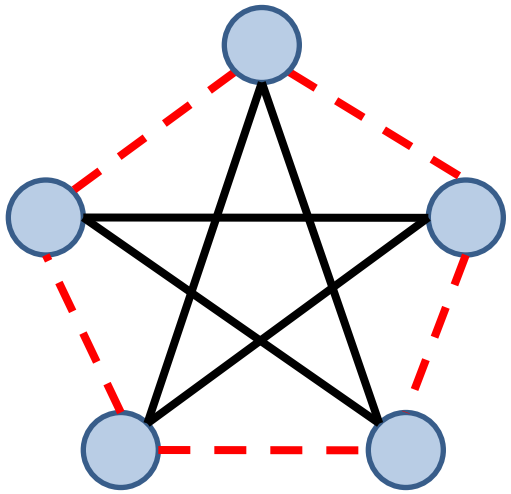


Tipos de Grafos

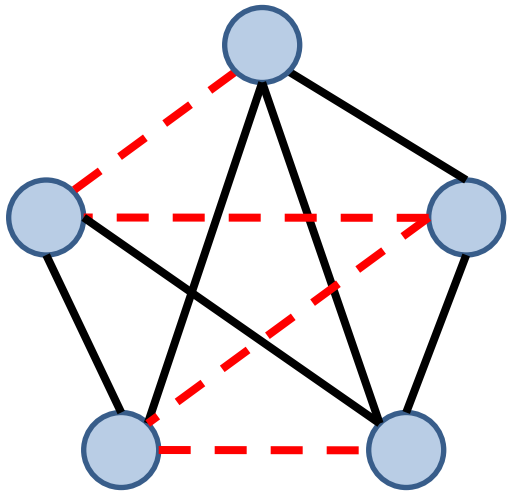
- Grafo Hamiltoniano
  - Grafo que possui um caminho que visita todos os seus vértices apenas uma vez.
  - Pode ser um ciclo



GRAFO HAMILTONIANO



CICLO HAMILTONIANO



CAMINHO HAMILTONIANO

ESTRUTURA DE DADOS: GRAFOS

Implementação

Prof. Jean Nunes

Tipos de representação

- Como representar um grafo no computador?
  - Existem duas abordagens muito utilizadas:
    - Matriz de Adjacência
    - Lista de Adjacência
  - Qual a representação que deve ser utilizada?
    - Depende da aplicação!

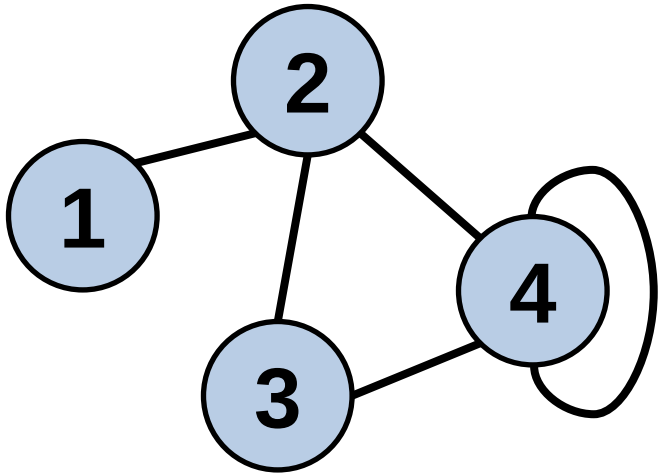
Tipos de representação

- Matriz de adjacência
  - Utiliza uma matriz **N x N** para armazenar o grafo, onde **N** é o número de vértices
    - Alto custo computacional, $O(N^2)$
  - Uma aresta é representada por uma marca na posição (i, j) da matriz
    - Aresta liga o vértice **i** ao **j**

Tipos de representação

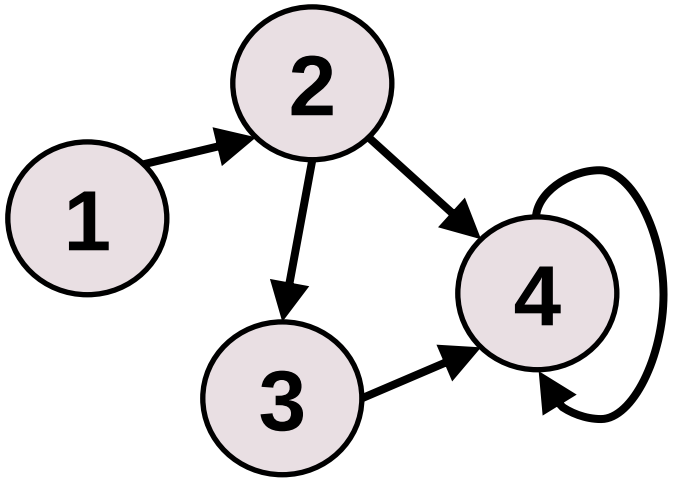
- Matriz de adjacência

GRAFO



	1	2	3	4
1	0	1	0	0
2	1	0	1	1
3	0	1	0	1
4	0	1	1	1

DIGRAFO



	1	2	3	4
1	0	1	0	0
2	0	0	1	1
3	0	0	0	1
4	0	0	0	1

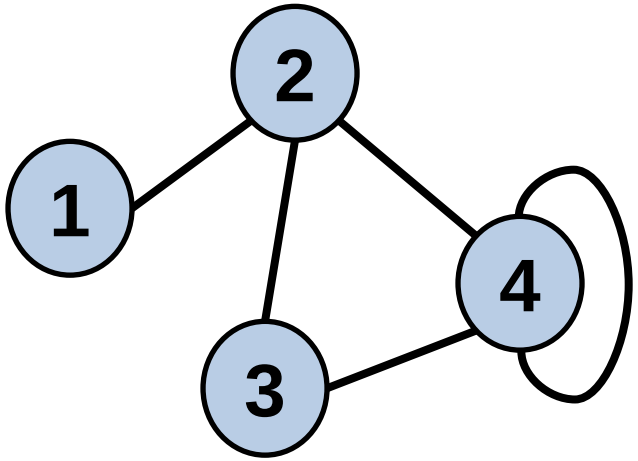
Tipos de representação

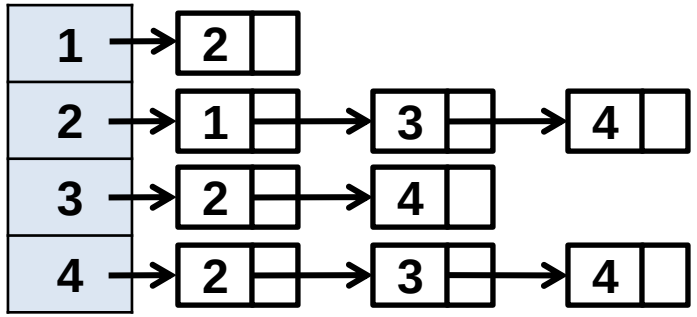
- Lista de adjacência
  - Utiliza uma lista para descrever as relações entre os vértices.
    - Um grafo contendo **N** vértices utiliza um *array* de ponteiros de tamanho **N** para armazenar os vértices do grafo
    - Para cada vértice é criada uma lista de arestas, onde cada posição da lista armazena o índice do vértice a qual aquele vértice se conecta

Tipos de representação

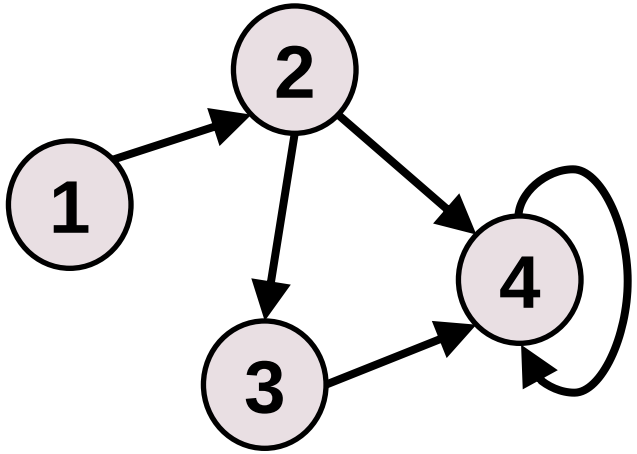
- Lista de adjacência

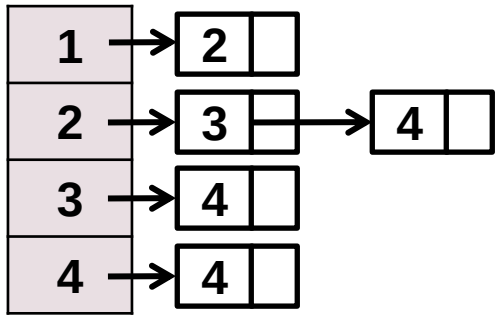
GRAFO





DIGRAFO





Tipos de representação

- Qual representação utilizar?
  - Lista de adjacência é mais indicada para um grafo que possui muitos vértices mas poucas arestas ligando esses vértices.
  - A medida que o número de arestas cresce e não havendo nenhuma outra informação associada a aresta (por exemplo, seu peso), o uso de uma matriz de adjacência se torna mais eficiente

Grafo: Implementando em C

- Vamos usar uma **lista de adjacência**
  - Lista de arestas: lista dinâmica encadeada

```c
//Arquivo Grafo.h
typedef struct grafo Grafo;
typedef struct vertice Vertice;
```

```c
// Grafo.c
// Estrutura para representar um
// vertice na lista de adjacência
struct vertice {
    int destino;
    float peso;
    struct vertice* prox;
};

struct listaArestas{
    struct vertice* head;
};
```

```c
// Grafo.c
// Definicao do tipo Grafo
struct grafo{
    int eh_ponderado;
    int nro_vertices;
    int grau_max;
    struct listaArestas* arestas;
    int* grau;
};
```

Tamanho das listas

Array de listas

Qtd de elementos em cada lista

Grafo: Implementando em C

- Criando um grafo

```c
//Arquivo Grafo.h
Grafo* cria_Grafo(int nro_vertices,
                  int grau_max,
                  int eh_ponderado);
```

```c
//main.c
Grafo* gr = cria_Grafo(10, 7, 0);
```

Grafo: Implementando em C

- Criando um grafo

```c
// Grafo.c
Grafo* cria_Grafo(int nro_vertices, int grau_max, int eh_ponderado){
    Grafo *gr;
    gr = (Grafo*) malloc(sizeof(struct grafo));
    if(gr != NULL){
        int i;
        gr->nro_vertices    = nro_vertices;
        gr->grau_max        = grau_max;
        gr->eh_ponderado    = eh_ponderado;
        gr->grau = (int*) calloc(nro_vertices, sizeof(int));

        gr->arestas = (struct listaArestas*) malloc(nro_vertices * sizeof(struct listaArestas));
        for(i=0; i<nro_vertices; i++)
            gr->arestas[i].head = NULL;
    }
    return gr;
}
```

Cria matriz arestas

Grafo: Implementando em C

```c
// Grafo.c
// Definicao do tipo Grafo
struct grafo{
    int eh_ponderado;
    int nro_vertices;
    int grau_max;
    struct listaArestas* arestas;
    int* grau;
};
```

```c
//main.c
Grafo* gr = cria_Grafo(10, 7, 0);
```

0	0
1	0
2	0
3	0
4	0
5	0
6	0
7	0
8	0
9	0

	0	1	2	3	4	5	6
0							
1							
2							
3							
4							
5							
6							
7							
8							
9							

Cria um grafo de 10 vértices.
Cada vértice se conecta com até outros 7 vértices

- Matriz 10x7 para as arestas
- Vetor “grau” guarda o número de conexões de cada um dos 10 vértices

Grafo: Implementando em C

- Liberando o grafo

```c
void libera_Grafo(Grafo* gr){
    if(gr != NULL){
        int i;
        for(i=0; i<gr->nro_vertices; i++){
            Vertice* temp = gr->arestas[i].head;
            while (temp != NULL) {
                Vertice* proximo = temp->prox;
                free(temp);
                temp = proximo;
            }
        }
        free(gr->arestas);
        free(gr->grau);
        free(gr);
    }
}
```

Libera matriz arestas representada pelas listas

Libera vetor de vértices, vetor de grau e grafo

Grafo: Implementando em C

- Inserindo uma aresta

```c
//Arquivo Grafo.h
int insereAresta(Grafo* gr, int orig, int dest, int eh_digrafo, float peso);
```

```c
// main.c
insereAresta(gr, 0, 1, 0, 3);
insereAresta(gr, 1, 3, 0, 2);
insereAresta(gr, 3, 2, 0, 4);
insereAresta(gr, 6, 1, 0, 1);
```

Grafo: Implementando em C

- Inserindo uma aresta

```c
// Grafo.c
int insereAresta(Grafo* gr, int orig, int dest, int eh_digrafo, float peso){
    if(gr == NULL)
        return 0;
    if(orig < 0 || orig >= gr->nro_vertices)
        return 0;
    if(dest < 0 || dest >= gr->nro_vertices)
        return 0;

    Vertice* novoVert = criarVertice(dest);
    novoVert->prox = gr->arestas[orig].head;
    if(gr->eh_ponderado)
        novoVert->peso = peso;
    gr->arestas[orig].head = novoVert;

    gr->grau[orig]++;

    if(eh_digrafo == 0)
        insereAresta(gr,dest,orig,1,peso);
    return 1;
}
```

Verifica se vértices existes

Insere no início da lista

Insere outra aresta se NÃO for dígrafo

Grafo: Implementando em C

- Inserindo uma aresta

```c
// Função para criar um novo vertice na lista de adjacência
Vertice* criarVertice(int destino) {
    Vertice* novoVertice    = (Vertice*) malloc(sizeof(struct vertice));
    if(novoVertice != NULL){
        novoVertice->destino    = destino;
        novoVertice->prox       = NULL;
    }
    return novoVertice;
}
```

Cria vértice de destino

Grafo: Implementando em C

```
insereAresta(gr,0,1,0,0);
```

Antes da inserção

0	0
1	0
2	0
3	0
4	0
5	0
6	0
7	0
8	0
9	0

	0	1	2	3	4	5	6
0							
1							
2							
3							
4							
5							
6							
7							
8							
9							

Após a inserção

0	1
1	1
2	0
3	0
4	0
5	0
6	0
7	0
8	0
9	0

	0	1	2	3	4	5	6
0	1						
1	0						
2							
3							
4							
5							
6							
7							
8							
9							

Grafo: Implementando em C

```
insereAresta(gr,0,1,0,0);
insereAresta(gr,1,3,0,0);
```

Antes da inserção

0	1
1	1
2	0
3	0
4	0
5	0
6	0
7	0
8	0
9	0

	0	1	2	3	4	5	6
0	1						
1	0						
2							
3							
4							
5							
6							
7							
8							
9							

Após a inserção

0	1
1	2
2	0
3	1
4	0
5	0
6	0
7	0
8	0
9	0

	0	1	2	3	4	5	6
0	1						
1	3	0					
2							
3	1						
4							
5							
6							
7							
8							
9							

Grafo: Implementando em C

```
insereAresta(gr,0,1,0,0);
insereAresta(gr,1,3,0,0);
insereAresta(gr,3,2,0,0);
```

Antes da inserção

0	1
1	2
2	0
3	1
4	0
5	0
6	0
7	0
8	0
9	0

	0	1	2	3	4	5	6
0	1						
1	3	0					
2							
3	1						
4							
5							
6							
7							
8							
9							

Após a inserção

0	1
1	2
2	1
3	2
4	0
5	0
6	0
7	0
8	0
9	0

	0	1	2	3	4	5	6
0	1						
1	3	0					
2	3						
3	2	1					
4							
5							
6							
7							
8							
9							

Grafo: Implementando em C

```
insereAresta(gr,0,1,0,0);
insereAresta(gr,1,3,0,0);
insereAresta(gr,3,2,0,0);
insereAresta(gr,6,1,0,0);
```

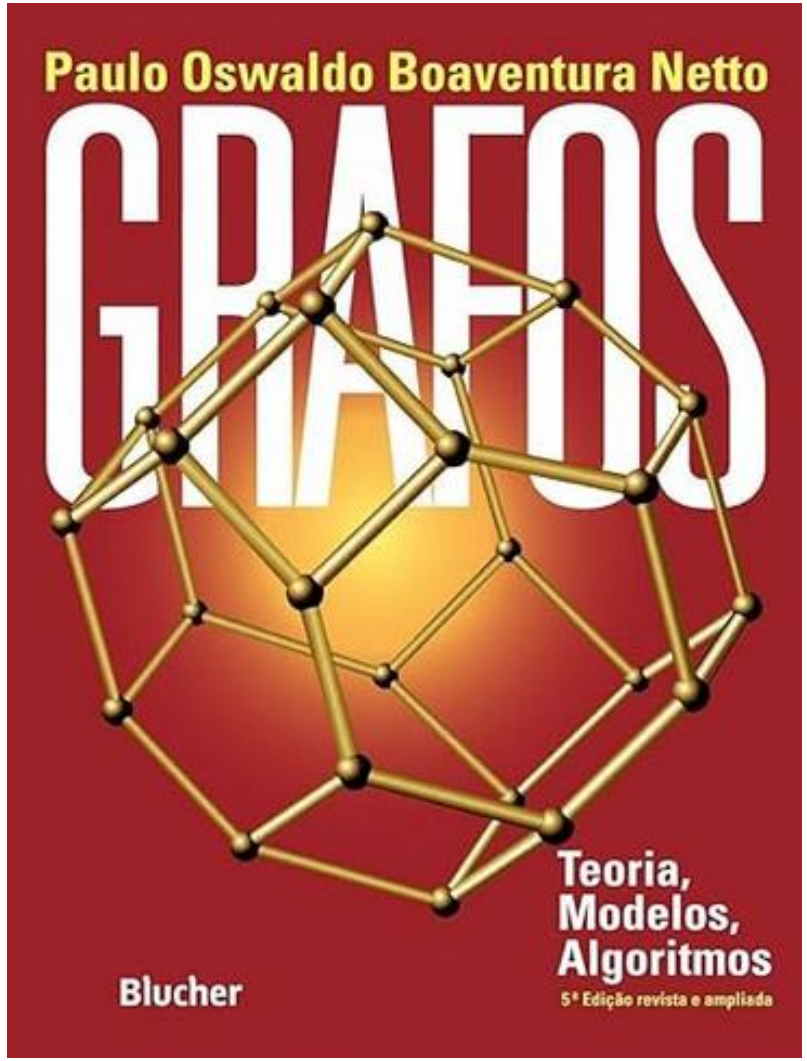
Antes da inserção

0	1
1	2
2	1
3	2
4	0
5	0
6	0
7	0
8	0
9	0

	0	1	2	3	4	5	6
0	1						
1	3	0					
2	3						
3	2	1					
4							
5							
6							
7							
8							
9							

Após a inserção

0	1
1	3
2	1
3	2
4	0
5	0
6	1
7	0
8	0
9	0

	0	1	2	3	4	5	6
0	1						
1	6	3	0				
2	3						
3	2	1					
4							
5							
6	1						
7							
8							
9							

Referências